



In association with



PROJECT TITLE- Capstone Project
Event Management System
(Microservices Architecture)

SUBMITTED BY:

NAINCY PANDEY

INTERN: T&T(EAD)

nainpandey.ext@deloitte.com

Date: March 29, 2026

Introduction:

EventZen is a scalable and modern Event Management System developed using Microservices Architecture. The application helps users to manage event venues, bookings, and attendees efficiently through a centralized platform.

The system is designed to provide seamless interaction between users, administrators, and event organizers. It allows users to register, login, browse venues, and book events while administrators can manage venues and monitor bookings.

The project demonstrates integration of multiple backend services using REST APIs and containerization using Docker. The application is deployed locally using Docker containers and can also be deployed on cloud platforms such as AWS for real-world accessibility. EventZen focuses on modular design, scalability, and security, ensuring flexibility for future enhancements.

Objectives:

- To design and develop a scalable event management platform using a microservices-based architecture.
- To provide a modular structure consisting of three specialized backend services: Auth Service, Venue Service, and Attendee Service.
- To demonstrate seamless backend and frontend integration using standardized RESTful APIs and an Nginx Gateway.
- To ensure data integrity and persistence using a combination of relational (MySQL) and document-based (MongoDB) databases.
- To implement a fully containerized deployment workflow using Docker and Docker Compose for high portability.

Technologies Used:

- **Frontend:** ReactJS (Vite), Tailwind CSS, Axios.
 - **Backend Services:**
 - Java Spring Boot (Authentication & Authorization).
 - ASP.NET Core 8 (Venue Infrastructure Management).
 - Node.js Express (Attendee Booking Logic).
 - **Gateways:** Nginx Reverse Proxy.
 - **Databases:** MySQL (User Data) and MongoDB (Booking Records).
 - **DevOps:** Docker, Docker Compose, AWS EC2
 - **Version Control:** GitHub
-

PROJECT STRUCTURE OVERVIEW

Project Structure:

Frontend (React.js):

- src/components/: Contains reusable UI components like the Navigation Bar and Modal system.
- src/pages/: Holds individual page components including the Venue Explorer, Booking Portfolio, and Admin Console.
- Nginx Proxy: Acts as the entry point, routing requests to appropriate backend services.

Backend Microservices:

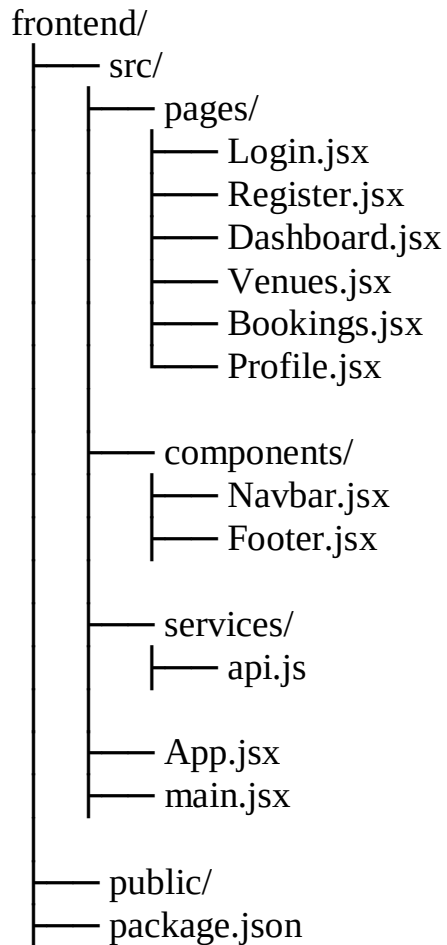
- Controller Layer: Manages API endpoints for Auth, Venues, and Bookings.
 - Service Layer: Encapsulates core business logic, such as ticket count validation and JWT generation.
 - Repository Layer: Handles direct interactions with MySQL (via JPA) and the MongoDB Atlas Cloud Cluster (via Mongoose).
 - DTOs/Models: Ensures structured request/response mapping across the polyglot environment.
-

Frontend Structure

Framework: React.js

The frontend is responsible for user interaction and UI rendering.

Folder Structure:



Explanation:

pages → contains all UI screens

components → reusable UI components

services → API calling functions

App.jsx → main application routing

main.jsx → entry point of React app

Backend Structure

The backend consists of 3 independent microservices:

1. **Auth Service**
2. **Venue Service**
3. **Attendee Service**

Each service has separate codebase and Docker container.

1. Auth Service (Spring Boot)

Handles user login and registration using JWT authentication.

Structure:

```
auth-service/  
├── controller/  
├── service/  
├── repository/  
├── model/  
├── security/  
└── application.properties
```

Explanation:

controller → API endpoints

service → business logic

repository → database communication

model → entity classes

security → JWT authentication configuration

2. Venue Service (.NET Core)

Handles venue management.

Structure:

```
venue-service/  
├── Controllers/  
├── Models/  
├── Data/  
└── Program.cs
```

Explanation:

Controllers → API endpoints

Models → venue data structure

Data → database connection

3. Attendee Service (Node.js)

Handles booking and attendee management.

Structure:

```
attendee-service/  
├── routes/  
├── models/  
├── controllers/  
└── server.js
```

Explanation:

routes → API endpoints

models → database schema

controllers → logic implementation

server.js → entry point

Docker Configuration

Docker is used to containerize each microservice so that the application runs consistently across different environments.

Files used:

docker-compose.yml → defines all services

Dockerfile → present in each service folder

.env → stores environment variables

Each service runs in separate container:

- frontend container
- auth-service container
- venue-service container
- attendee-service container
- mysql container

Benefits:

- Easy deployment
- Platform independent
- Consistent environment
- Easy scaling

GITHUB LINK: <https://github.com/naincypandey/EventZen-Capstone-AWS>

FEATURES & MODULES

1. User Authentication Module
2. Venue Management Module
3. Attendee / Booking Module

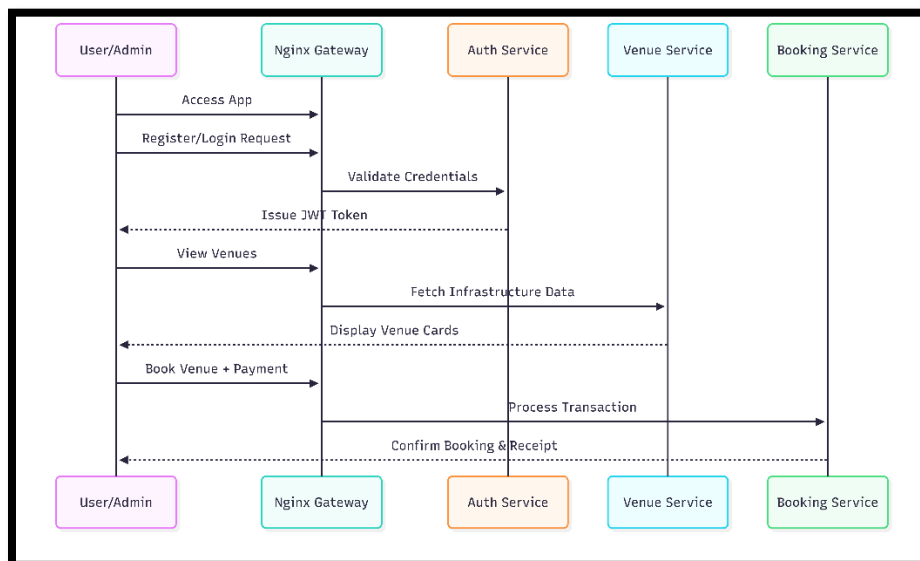
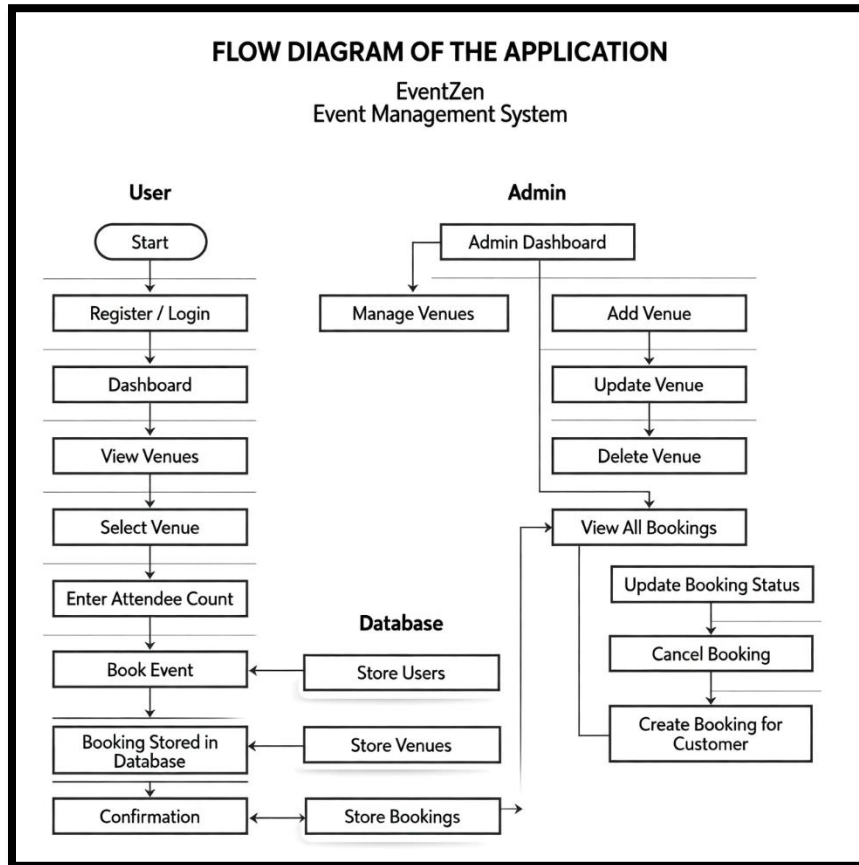
Features of EventZen

EventZen provides a centralized platform for managing event venues and bookings. The system supports both users and administrators to perform various operations efficiently.

Main Features:

- User Registration and Login using secure authentication
 - Browse available venues
 - Book events or venues
 - Manage attendee details
 - Admin can add, update, or delete venues
 - REST API integration between frontend and backend
 - Microservices architecture for scalability
 - Docker-based deployment
 - Cloud deployment using AWS
-

FLOW DIAGRAM



USER AUTHENTICATION

The Authentication Module ensures secure login and registration of users.

Key Features:

User Registration:

- New users can create account using email and password
- User details stored securely in database

User Login:

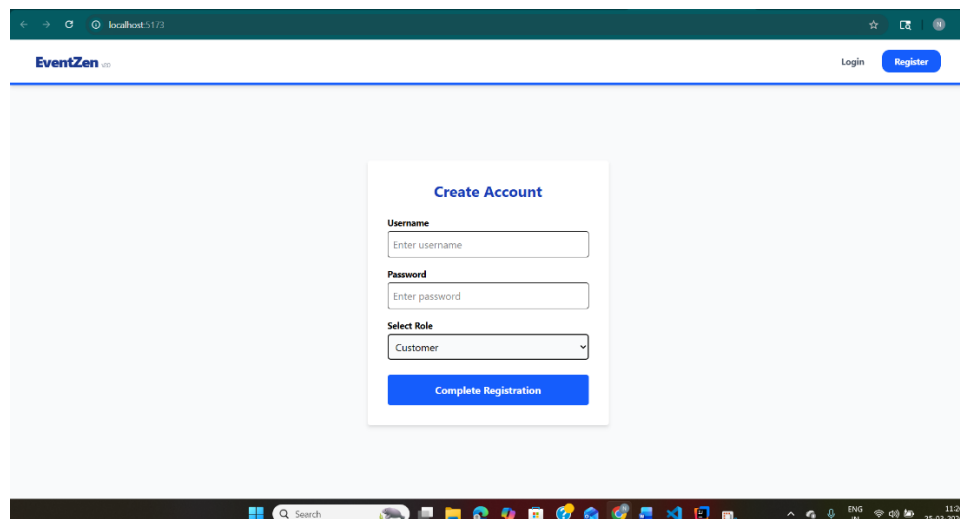
- Registered users can login using credentials
- JWT token used for authentication

Security:

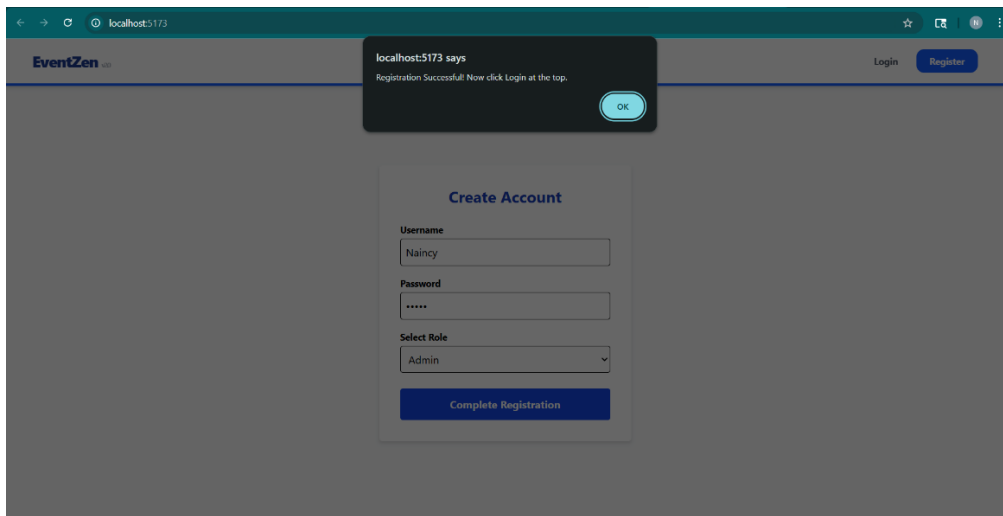
- Password stored securely
- Unauthorized users cannot access protected routes

Benefits:

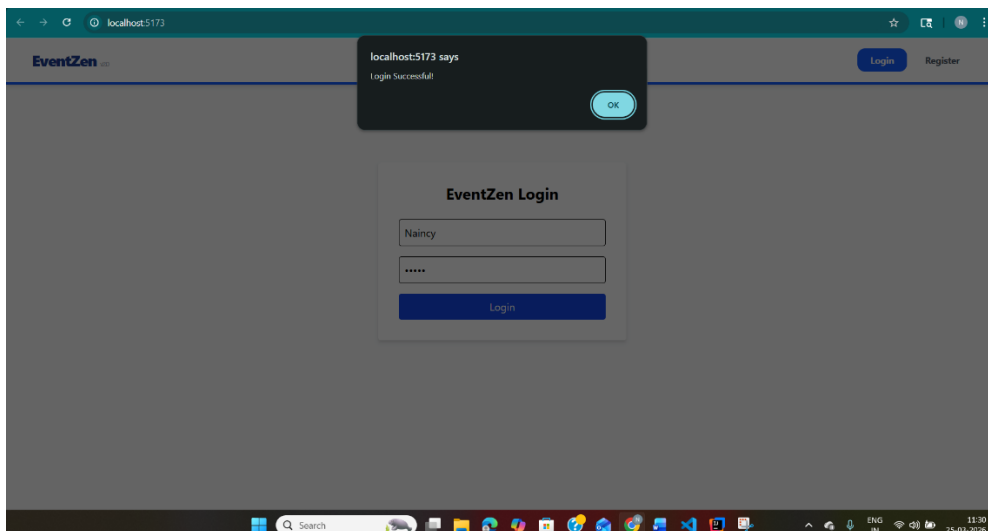
- Ensures data security
- Provides secure access to system
- Protects user information



Register page



Registrations success



Login success

VENUE MANAGEMENT

The Venue Module allows administrators to manage event venues.

Key Features:

Add Venue:

- Admin can add new venues with details
- Includes venue name, location, capacity

Update Venue:

- Admin can update venue details
- Allows modification of venue information

Delete Venue:

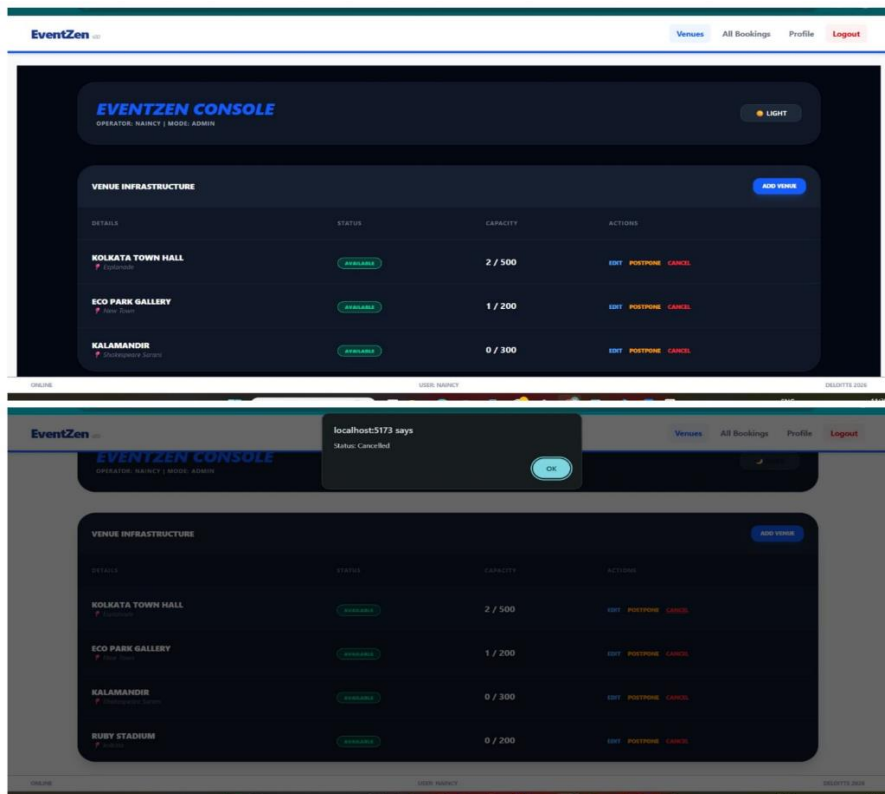
- Admin can remove venues if not required

View Venue:

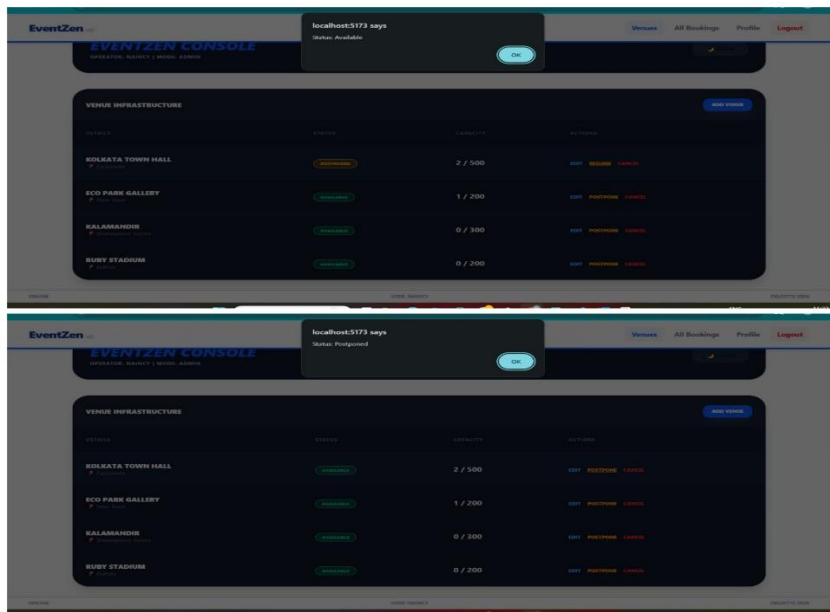
- Users can browse available venues

Benefits:

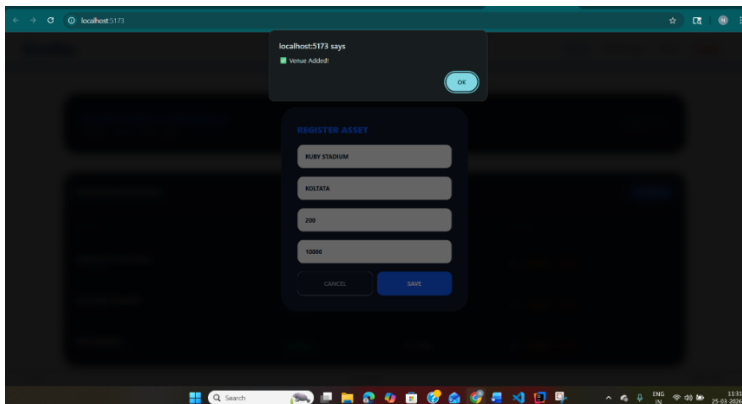
- Easy venue management
- Organized venue information
- Improves event planning efficiency



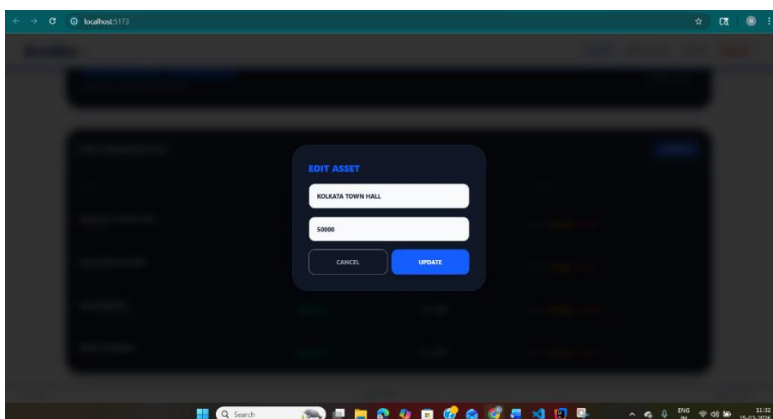
**Amin Dashboard
with venue
managements**



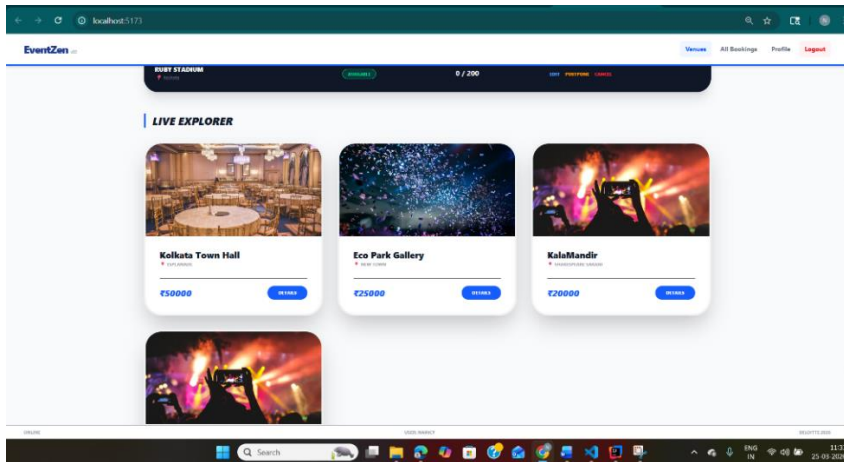
Amin Dashboard with venue managements



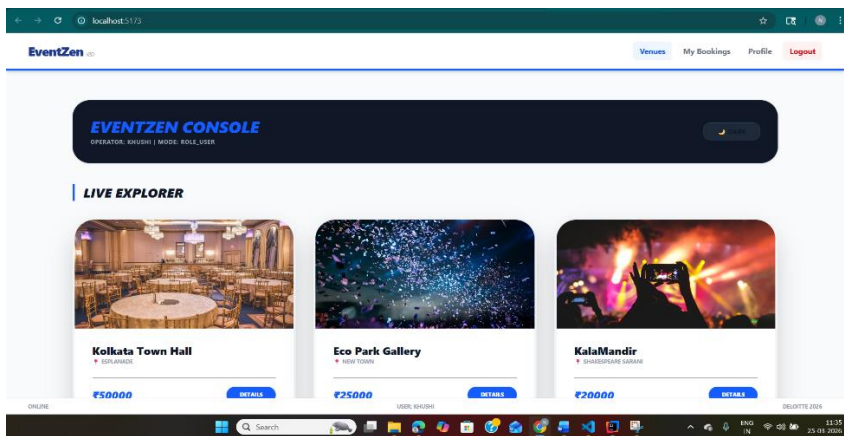
Successfully venue & event updating



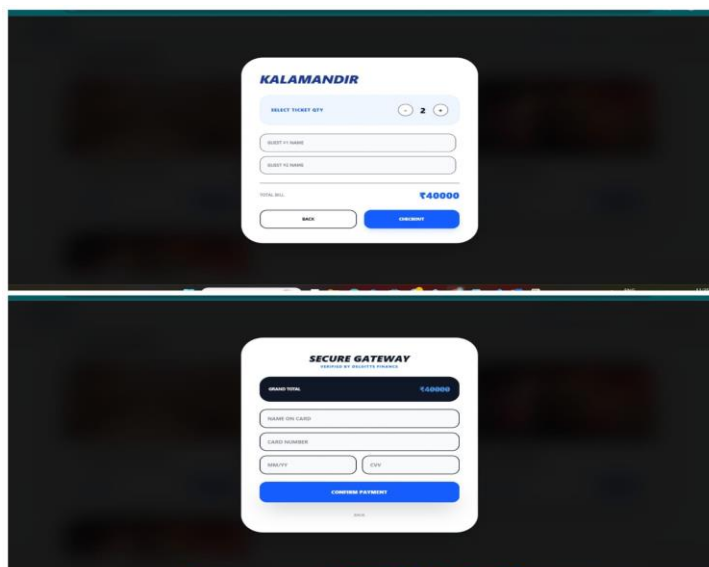
Event Edit



Admin viewing event detail with booking facility available



Customer Dashboard view and book Events



Booking and payment of an Event

Payment Detail

ATTENDEE MANAGEMENT

The Attendee Module manages booking information for events.

Key Features:

Book Event:

- Users can select venue and create booking

Manage Attendees:

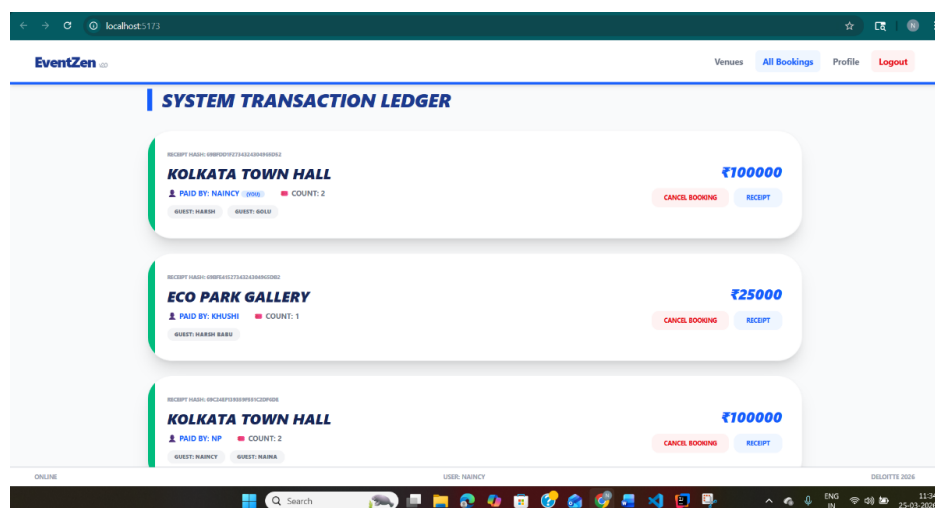
- Stores attendee information
- Displays booking details

View Bookings:

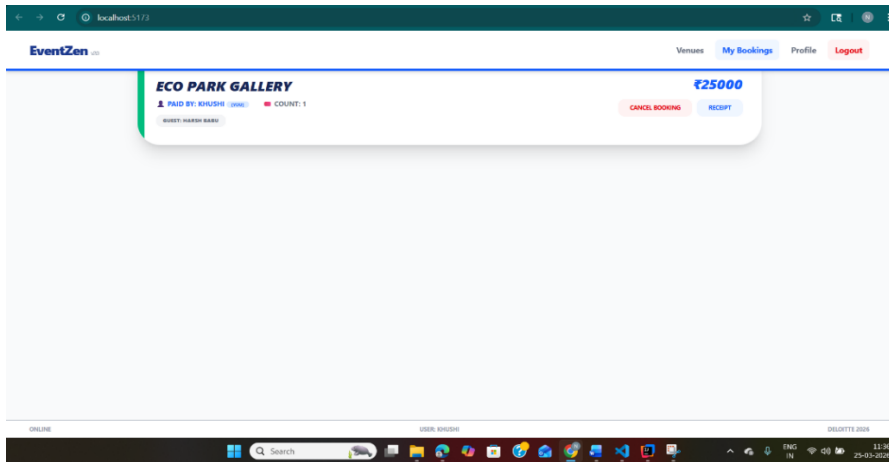
- Users can see their booking history

Benefits:

- Simplifies booking process
- Maintains attendee records
- Provides organized booking system



Admin Transaction ledger



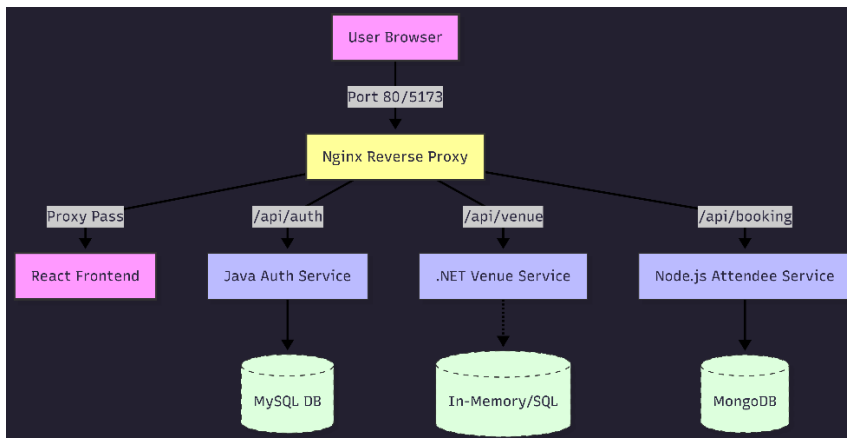
**Customer
booking records**

SYSTEM ARCHITECTURE

EventZen follows a Microservices Architecture where each service is independently developed and deployed.

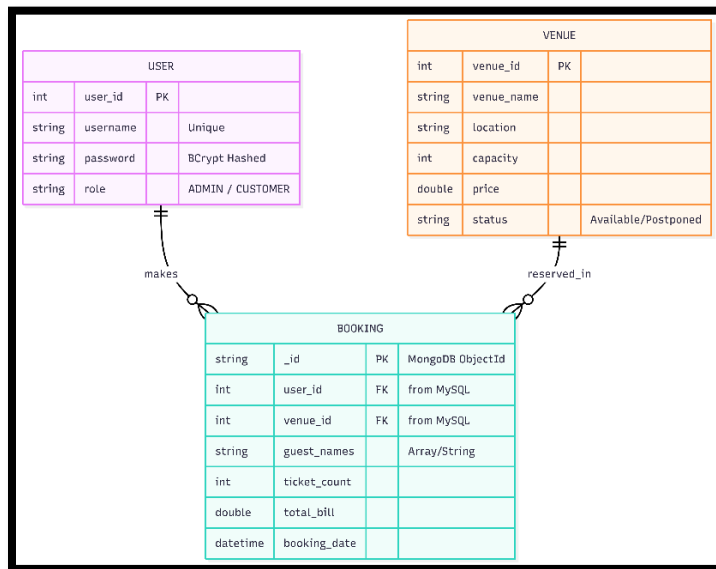
Architecture Description:

- React Frontend communicates with backend services via Nginx Gateway.
- Auth Service (Java) handles login and user persistence in MySQL.
- Venue Service (.NET) manages venue operations.
- Attendee Service (Node.js) manages bookings and attendee details in MongoDB Atlas.
- All services are containerized using Docker for a consistent environment.



ER DIAGRAM

This diagram shows database tables and their relationships.



WIREFRAMES

The UI for EventZen was designed to be clean and responsive, ensuring that both Admins and Customers can navigate the event infrastructure without confusion.

Key Design Elements:

- **Admin Console:** A dark-themed dashboard designed for high-density information display.
- **Secure Gateway:** A minimalist modal for payment processing that prioritizes user trust and data security.
- **Live Explorer:** A grid-based layout for venue cards that adapts to various screen sizes for a consistent experience.

EventZen

Logo | Login | Register

Login

Username

Password

Login

EventZEN

Venues My Bookings Profile Logout

EVENTZEN CONSOLE

+ Add Venue

Name	Status	Capacity	Actions
Venue 1	Active	2 / 500	Edit: x
Venue 2	Active	1 / 200	Edit: x

+ Add Venue

EventZen

Logo | Login | Register

ADD / EDIT VENUE

Venue Name

Location

Price

Capacity

Cancel Save

EventZen

Logo | Login | Register

Book Venue

Sticket

- 1 + XXXX

First Name

Total: ₹XXXX Checkout

EventZEN

Venues My Bookings Profile Logout

Venues

Venue 1

₹5,000

Book

Venue 2

₹5,000

Book

Venue 2

₹5,600

Book

EventZEN

Venues My Bookings Profile Logout

Payment

Card Name

MM / YY

CVV

Total: ₹XXXX Back Checkout

EVENTZEN

PROFILE

Name

Status: Active

Logout

EventZEN

Venues My Bookings Profile Logout

Booking Ledger

Venue 1

Date: ₹XXXX

Cancel Booking

Venue 2

Date: ₹XXXX

Cancel Booking

Receipt

API ENDPOINTS & SWAGGER DOCUMENTATION

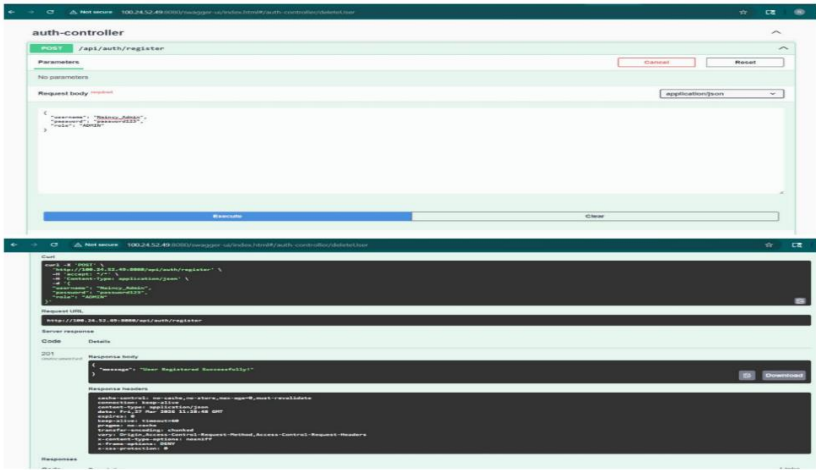
The application uses RESTful APIs to communicate between the React Frontend and Backend Microservices.

Endpoint	Method	Description
/api/auth/register	POST	Create a new user or admin account.
/api/auth/login	POST	Authenticate user and return JWT token.
/api/auth/users	GET	Fetch all registered users for admin audit.
/api/venue	GET	Retrieve list of all available event venues.
/api/booking	POST	Save new booking and attendee details to Atlas.

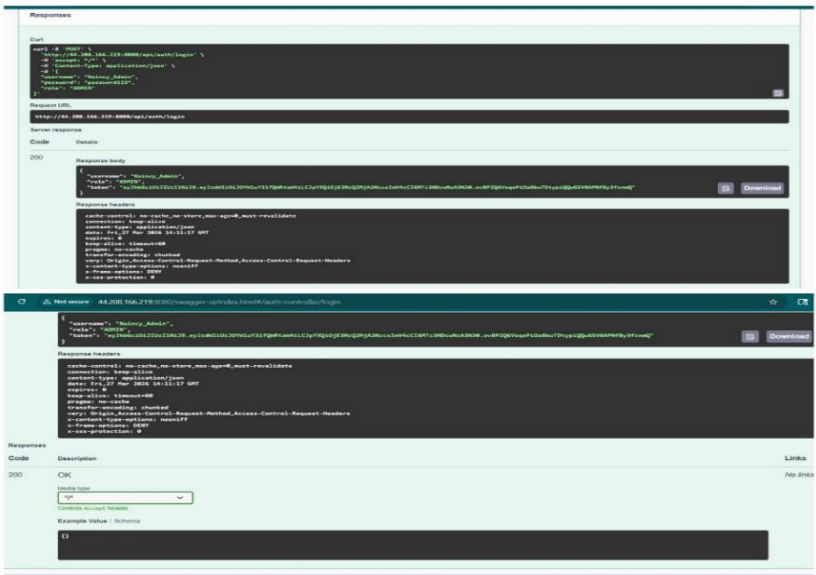
Swagger UI Features:

- Interactive documentation for all microservices.
- Real-time testing of API responses and status codes.
- JWT token-based endpoint testing.

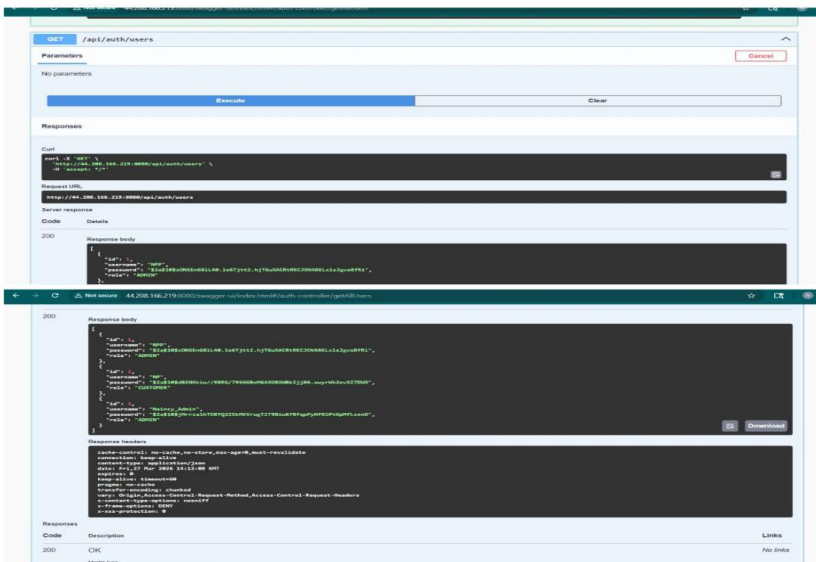
CAPSTONE



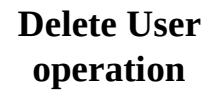
Registration Post operation



Login Post operation

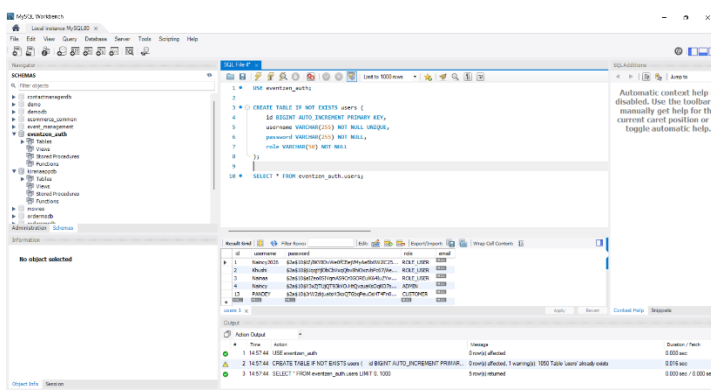


User Get operation



To verify "Database schema alignment," we performed direct queries on the MySQL instance.

- User Table: Verified that usernames and encrypted passwords are saved correctly.
- Role Mapping: Confirmed that roles (ADMIN/CUSTOMER) are properly persisted.



DEPLOYMENT & DATABASE VERIFICATION

The application is fully containerized and deployed on the AWS Cloud.

Dockerization:

- Separate Dockerfiles for Frontend, Java, .NET, and Node.js services.
- Docker Compose manages the multi-container orchestration.
- Ensures "Write Once, Run Anywhere" consistency across developer and tester environments.

Cloud Infrastructure:

- AWS EC2: Hosts the Docker engine and Nginx gateway.
- MongoDB Atlas: Provides a scalable cloud database for attendee records.
- Nginx: Handles reverse proxying to route traffic to specific containers.

Database Verification:

- MySQL: Verified using MySQL Workbench showing user persistence.
- MongoDB: Verified via Atlas Compass/Terminal showing booking logs.

```

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-0f7b5616ca0fb2783&osUser=ec2-user&sshPort=22&addressFamily...
AWS Search [Alt+S] United States (N. Virginia) Account ID: 0475-9358-4079 mainpandey.ext@deloitte.com

=> [attendee-service] resolving provenance for metadata file 0.0s
=> [frontend build 5/6] COPY . . 0.1s
=> [frontend build 6/6] RUN npm run build 4.5s
=> [frontend stage-1 3/3] COPY --from=build /app/dist /usr/share/nginx/html 0.1s
=> [frontend] exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:ec9f90f613f5be2a131fd515e863e09e1f434d19e271bdde9b42036c9024170d 0.0s
=> => naming to docker.io/library/eventzen-capstone-aws-frontend 0.0s
=> [frontend] resolving provenance for metadata file 0.0s
=> [venue-service stage-1 3/3] COPY --from=build /app/out . 0.1s
=> [venue-service] exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:643c0201b853bc9ee8671b7e6e952d411b5cc8772daabd437c8f8b34bb28621 0.0s
=> => naming to docker.io/library/eventzen-capstone-aws-venue-service 0.0s
=> [venue-service] resolving provenance for metadata file 0.0s
[+] up 11/11
✔ Image eventzen-capstone-aws-auth-service Built 105.7s
✔ Image eventzen-capstone-aws-venue-service Built 105.7s
✔ Image eventzen-capstone-aws-attendee-service Built 105.7s
✔ Image eventzen-capstone-aws-frontend Built 105.7s
✔ Network eventzen-capstone-aws_default Created 0.2s
✔ Container mongo-db Started 1.0s
✔ Container mysql-db Healthy 21.7s
✔ Container venue-service Started 22.3s
✔ Container auth-service Started 22.3s
✔ Container attendee-service Started 1.3s
✔ Container frontend-react Started 22.7s
[ec2-user@ip-172-31-83-170 EventZen-Capstone-AWS]$

i-0f7b5616ca0fb2783 (Test)
PublicIPs: 44.208.166.219 PrivateIPs: 172.31.83.170

```

PROJECT RESOURCES & DEPLOYMENT ENVIRONMENTS

The EventZen system is fully containerized and was successfully deployed and verified on an AWS EC2 instance. The following details reflect the environment where the final integration testing was performed.

1. AWS Deployment & Verification Details

- Verified Public IP: 44.208.166.219
- Frontend UI: <http://44.208.166.219:5173>
- Auth Service (Java) Swagger: <http://44.208.166.219:8080/swagger-ui/index.html>
- Venue Service (.NET) Swagger: <http://44.208.166.219:5059/swagger/index.html>

Note: The instance was hosted for deployment validation and capturing architectural evidence. While the instance is currently stopped to manage cloud resources, the full system orchestration is verified in the terminal logs on Page 21 ([+] up 11/11).

2. Cloud Database Architecture (MongoDB Atlas)

To ensure high availability beyond the lifecycle of the EC2 instance, the Attendee / Booking Module is integrated with a MongoDB Atlas Cloud Cluster.

- Cluster Type: M0 Sandbox (Shared)
- Data Persistence: Verified via real-time document insertion during the booking flow. Data remains persistent in the cloud even when the application containers are stopped.

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	Action
sgr-010ee563bf186d610	HTTPS	TCP	443	Custom	Q	Delete
sgr-0e5bb2ccb1275adcb	Custom TCP	TCP	8080	Custom	Q	Delete
sgr-0d5d5dec1af56bdda	Custom TCP	TCP	5059	Custom	Q	Delete
sgr-0e9fd72c421e34038	Custom TCP	TCP	5001	Custom	Q	Delete
sgr-0ceaa6bf4043ae671	Custom TCP	TCP	5173	Custom	Q	Delete
sgr-03f24e1996ce8769a	HTTP	TCP	80	Custom	Q	Delete
sgr-002ff5fa57294a7aa	SSH	TCP	22	Custom	Q	Delete

CONCLUSION

The EventZen Event Management System successfully fulfills the requirements of the Capstone Project by implementing a robust, polyglot microservices architecture.

By deploying the application on the AWS EC2 environment, the project demonstrates a complete "Code-to-Cloud" workflow. The integration of Java (Spring Boot), .NET 8, and Node.js proves that a diverse technology stack can be harmonized into a single, high-performance platform using Docker Compose and Nginx Reverse Proxying.

The system effectively handles:

1. Secure Authentication: Protecting user data with JWT and BCrypt.
 2. Scalable Data Management: Using a hybrid approach with MySQL and MongoDB Atlas.
 3. Enterprise-Grade Deployment: Utilizing Docker to ensure the user can deploy and verify the application with minimal configuration.
-